

SEBA-XAI Prototype

Five-Unit Full Syllabus

Project: Secure Explainable Blockchain-Audited Access Governance Prototype

Level: M.Tech / early research project

Date: 2026-05-28

This PDF is generated from `SEBA_XAI_Prototype_5_Unit_Syllabus.md`.

SEBA-XAI Prototype: 5-Unit Full Syllabus

Course type: Research implementation syllabus
Level: M.Tech / early research project
Project: SEBA-XAI - Secure Explainable Blockchain-Audited Access Governance Prototype
Date: 2026-05-28

Course Purpose

This syllabus is designed to help a student learn the complete SEBA-XAI prototype in a structured way. It focuses on the actual code, experiments, outputs, and research reasoning already present in this repository.

The prototype studies secure, explainable, and auditable access governance for sensitive police-style records. It does not use real police data and does not replace CCTNS or ICJS. It is a synthetic research prototype for testing the idea of a secure overlay.

The full prototype has two implementation layers:

- 1. `prototype/synthetic_access_sim/` - the original step-by-step simulator.
- 2. `src/seba/` - the research-grade package with schema, attacks, baselines, scoring, NS-PI, and tests.

Course Objectives

By the end of this syllabus, the student should be able to:

- 1. Explain the complete SEBA-XAI prototype flow from synthetic request generation to paper evidence tables.
- 2. Understand how policy rules validate access requests.
- 3. Explain where XAI is used in the prototype and why it matters.
- 4. Explain why raw sensitive records stay off-chain.
- 5. Compare mutable logs, signed hash chains, and permissioned blockchain-style audit logs.
- 6. Understand how latency, storage, tamper detection, metadata exposure, and policy ablations are measured.
- 7. Understand the ADV-AUDIT attack catalog and baseline defenses.
- 8. Explain NS-PI, drift detection, and counterfactual explanations.
- 9. Identify the current research limitation and the next required experiment: `compromised_signer`.

Course Outcomes

After completing the five units, the student should be able to:

Outcome	Description
CO1	Describe the SEBA-XAI architecture and its research boundary.

CO2	Generate and inspect synthetic police-style access requests.
CO3	Trace an access request through policy validation and explanation generation.
CO4	Explain audit integrity using mutable logs, signed hash chains, and permissioned blockchain-style blocks.
CO5	Measure and interpret latency, storage, metadata exposure, and policy ablation results.
CO6	Explain the attack catalog, AAS scoring, NS-PI model, drift detector, and current research gap.

Prerequisites

The student should know the basics of:

- Python programming;
- CSV and JSON files;
- basic access control;
- basic hashing;
- basic blockchain concepts;
- basic machine learning terminology;
- command-line execution;
- reading Markdown research notes.

Before starting, run:

```
pip install -e ".[dev]"
make test
```

Recommended files to read first:

```
README.md
REPRODUCE.md
SESSION_HANDOFF.md
results/FINDINGS.md
CONTRIBUTION.md
```

Unit 1: Prototype Foundation, Research Scope, And Synthetic Data

Unit Goal

This unit explains what the prototype is, why it was built, what it does not claim, and how the synthetic access-request dataset is generated.

Learning Objectives

After this unit, the student should be able to:

1. Explain the difference between the original broad research idea and the narrowed SEBA-XAI direction.
2. Explain why the project is about access governance, not crime prediction.
3. Understand the meaning of a CCTNS/ICJS-compatible overlay.
4. Generate synthetic access requests.
5. Read the generated station, officer, case, record, and request files.

Topics

1.1 Research Problem

The broad starting idea was "AI for police and crime data in India." This was narrowed because general crime prediction can become unsafe, unsupported, and hard to publish without real sensitive data.

The safer and stronger research problem is:

Can we design and evaluate a secure, explainable, auditable access-governance overlay for sen.

The system is an overlay. This means it is designed as an additional governance layer above existing systems, not as a replacement for CCTNS or ICJS.

1.2 Important Boundaries

The prototype does not claim:

- real police deployment;
- legal compliance;
- production security;
- real CCTNS or ICJS integration;
- real FIR or police record processing;
- prediction of criminals;
- state-of-the-art crime prediction.

These boundaries are important because a research paper must not overclaim.

1.3 Synthetic Dataset Design

The prototype creates synthetic versions of:

- police stations;
- officers;
- cases;
- records;
- access requests.

Each access request represents a situation where a synthetic officer requests access to a synthetic sensitive record.

1.4 Main File

```
prototype/synthetic_access_sim/generate_synthetic_requests.py
```

1.5 Command

```
python3 prototype/synthetic_access_sim/generate_synthetic_requests.py \  
  --run-id unit1_synthetic_seed42 \  
  --seed 42 \  
  --num-requests 1000
```

1.6 Output Files

```
prototype/runs/unit1_synthetic_seed42/  
  config.yaml  
  metrics.json  
  artifacts/stations.csv  
  artifacts/officers.csv  
  artifacts/cases.csv  
  artifacts/records.csv  
  artifacts/access_requests.csv  
  artifacts/dataset_manifest.json  
  artifacts/data_dictionary.md  
  artifacts/dataset_profile.csv
```

1.7 Tiny Technical Details To Learn

- `seed` makes the run reproducible.
- `run_id` decides the output folder name.
- `num_requests` decides workload size.
- `access_requests.csv` is the main input for policy validation.
- The dataset is useful for workflow experiments, not for real crime analysis.

1.8 Hands-On Work

1. Run the command.
2. Open `access_requests.csv`.
3. Pick one request row.
4. Identify:
 - officer;
 - station;
 - requested record;
 - action;
 - purpose;
 - sensitivity level;
 - timestamp.

1.9 Student Notes To Prepare

Write one page answering:

1. What is one access request?

2. Why is synthetic data used?
3. Why does this project avoid real police data?
4. Why is this research about access governance instead of crime prediction?

1.10 Viva Questions

1. What does SEBA-XAI stand for?
2. What does "overlay" mean?
3. Why is CCTNS/ICJS not replaced?
4. What is synthetic data?
5. Why is a seed required?
6. What is the limitation of synthetic data?

Unit 2: Policy Validation, Access Control, And XAI

Unit Goal

This unit explains how a generated request is validated using policy rules and how explanations are created for decisions.

Learning Objectives

After this unit, the student should be able to:

1. Explain where request validation happens.
2. Understand allow, deny, and escalate decisions.
3. Explain how ABAC/PBAC ideas appear in the policy oracle.
4. Understand rule-trace explanations.
5. Understand counterfactual explanations in the NS-PI package.

Topics

2.1 Access Request Validation

After synthetic requests are generated, the policy oracle checks each request against policy rules.

The request can become:

- `allow` - access is permitted;
- `deny` - access is rejected;
- `escalate` - human or superior approval is required.

2.2 Main Policy Oracle File

```
prototype/synthetic_access_sim/policy_oracle.py
```

2.3 Policy Configuration File

```
prototype/synthetic_access_sim/policies/seba_xai_policy_v1.json
```

2.4 Command

```
python3 prototype/synthetic_access_sim/policy_oracle.py \  
--input-run-id unit1_synthetic_seed42 \  
--run-id unit2_policy_seed42
```

2.5 Output Files

```
prototype/runs/unit2_policy_seed42/  
config.yaml  
metrics.json  
artifacts/labeled_access_requests.csv  
artifacts/policy_summary.csv  
artifacts/explanation_artifacts.jsonl  
artifacts/policy_rules.json  
artifacts/dataset_manifest.json  
artifacts/data_dictionary.md
```

2.6 What The Oracle Adds

For every request, the policy oracle adds:

- decision;
- primary_reason_code;
- decisive_attributes;
- xai_explanation;
- decision_hash;
- explanation_hash;
- audit_anchor_hash.

2.7 ABAC And PBAC In This Prototype

ABAC means Attribute-Based Access Control. It checks attributes such as:

- subject attributes: officer role, station, assignment;
- object attributes: record type, sensitivity level, case jurisdiction;
- action attributes: read, update, approve;
- environment attributes: time, emergency flag, approval token;
- policy version.

PBAC means Policy-Based Access Control. It means the access decision follows explicit policy rules and policy versions.

In this prototype, the policy oracle behaves like a deterministic ABAC/PBAC decision engine.

2.8 XAI In This Unit

The first form of XAI is rule-trace explanation.

Example:

```
Denied because officer jurisdiction does not match record jurisdiction.
```

The second form of XAI appears in:

```
src/seba/nsapi/counterfactual.py
```

This creates counterfactual explanations.

Example:

```
Would have been allow if approval_token_status = PRESENT_VALID.
```

2.9 What XAI Does Not Prove

XAI does not prove:

- the decision is legally correct;
- the decision is morally correct;
- the system is unbiased;
- the system is ready for deployment.

XAI only helps explain how the decision was reached.

2.10 Tiny Technical Details To Learn

- `decision_hash` protects the decision artifact.
- `explanation_hash` protects the explanation artifact.
- `audit_anchor_hash` connects the decision and explanation to later audit logs.
- Explanations can be sensitive and may need access control too.

2.11 Hands-On Work

1. Run the policy oracle.
2. Open `labeled_access_requests.csv`.
3. Pick one `deny`, one `allow`, and one `escalate` row.
4. Read the reason code and explanation.
5. Explain each row in simple English.

2.12 Student Notes To Prepare

Write a two-column table:

Decision	Meaning in this prototype
----------	---------------------------

allow	
deny	
escalate	

Then write one example explanation for each.

2.13 Viva Questions

1. Where is a request validated?
2. What is the policy oracle?
3. What is the difference between ABAC and PBAC?
4. Why is `escalate` necessary?
5. Where does XAI appear?
6. Why should explanations be hashed?

Unit 3: Audit Logs, Hash Chains, Blockchain Layer, And Off-Chain Storage

Unit Goal

This unit explains the audit and blockchain part of the prototype. It also explains why raw sensitive records stay off-chain.

Learning Objectives

After this unit, the student should be able to:

1. Explain mutable logs.
2. Explain signed append-only hash chains.
3. Explain permissioned blockchain-style audit.
4. Explain why the prototype is closer to PoA/PBFT-style validation, not PoW or PoS.
5. Explain off-chain storage and on-chain commitments.

Topics

3.1 Mutable Log Baseline

A mutable log is a normal editable log. It is easy to store and query, but weak for tamper evidence.

Main file:

```
prototype/synthetic_access_sim/audit_baseline.py
```

Command:

```
python3 prototype/synthetic_access_sim/audit_baseline.py \  
  --input-run-id unit2_policy_seed42 \  
  --run-id unit3_audit_seed42
```

Important output:

```
prototype/runs/unit3_audit_seed42/artifacts/mutable_access_log.csv
```

3.2 Signed Append-Only Hash Chain

A signed hash chain links each row to the previous row using hashes.

Simple structure:

```
row 1 hash  
-> row 2 hash includes row 1 hash  
-> row 3 hash includes row 2 hash  
-> row 4 hash includes row 3 hash
```

If a row is edited, deleted, or reordered, verification should fail.

Important output:

```
prototype/runs/unit3_audit_seed42/artifacts/signed_hash_chain_log.csv  
prototype/runs/unit3_audit_seed42/artifacts/tamper_test_results.csv  
prototype/runs/unit3_audit_seed42/artifacts/audit_detection_summary.csv
```

3.3 Permissioned Blockchain-Style Audit

The blockchain-style audit layer groups audit events into blocks and uses known synthetic validators.

Main file:

```
prototype/synthetic_access_sim/blockchain_audit.py
```

Command:

```
python3 prototype/synthetic_access_sim/blockchain_audit.py \  
  --input-run-id unit3_audit_seed42 \  
  --run-id unit3_blockchain_seed42
```

Important outputs:

```
prototype/runs/unit3_blockchain_seed42/artifacts/permissioned_audit_blocks.jsonl  
prototype/runs/unit3_blockchain_seed42/artifacts/block_event_index.csv  
prototype/runs/unit3_blockchain_seed42/artifacts/validator_set.json  
prototype/runs/unit3_blockchain_seed42/artifacts/blockchain_tamper_test_results.csv  
prototype/runs/unit3_blockchain_seed42/artifacts/blockchain_detection_summary.csv
```

3.4 What Type Of Blockchain Is This?

This prototype simulates a permissioned blockchain-style audit layer.

It is not:

- Proof of Work;
- Proof of Stake;
- public blockchain;
- deployed Hyperledger Fabric.

It is closer to:

- Proof of Authority style known validators;
- PBFT-style quorum validation;
- consortium audit ledger design.

This is suitable for police-style systems because validators should be known agencies, not anonymous miners.

3.5 What Goes On-Chain

The ledger stores audit commitments such as:

- request ID;
- decision hash;
- explanation hash;
- policy version;
- model version;
- block hash;
- previous block hash;
- validator signatures or endorsements;
- record pointer hash.

3.6 What Stays Off-Chain

Raw sensitive records stay off-chain. This means the actual sensitive content remains in protected storage, not on the ledger.

Examples of off-chain content:

- FIR text;
- witness statements;
- victim records;
- juvenile records;
- forensic reports;
- evidence media;
- case diary material.

Main file:

```
prototype/synthetic_access_sim/offchain_storage.py
```

Command:

```
python3 prototype/synthetic_access_sim/offchain_storage.py \  
--input-run-id unit2_policy_seed42 \  
--run-id unit3_offchain_seed42
```

Simple flow:

```
raw sensitive record  
-> encrypted off-chain storage  
-> pointer/hash  
-> audit ledger commitment
```

3.7 Tiny Technical Details To Learn

- Mutable logs are weak but useful baselines.
- Signed hash chains detect edit/delete/reorder tampering.
- Blockchain-style audit adds block grouping and validator quorum.
- Raw records should not be placed on-chain because blockchain data is hard to remove.
- Even pointer metadata can leak information.

3.8 Hands-On Work

1. Run audit baseline.
2. Compare mutable log and signed hash-chain output.
3. Run blockchain audit.
4. Open `validator_set.json`.
5. Open `permissioned_audit_blocks.jsonl`.
6. Run off-chain storage simulation.
7. Explain what is stored on-chain and what stays off-chain.

3.9 Student Notes To Prepare

Draw three diagrams:

1. Mutable log.
2. Signed hash-chain.
3. Permissioned blockchain-style audit with off-chain storage.

3.10 Viva Questions

1. Why is mutable log not enough?
2. How does a hash chain detect tampering?
3. What is a validator?
4. What is quorum?
5. Why is this not PoW or PoS?
6. Why do raw records stay off-chain?
7. What is metadata leakage?

Unit 4: Experiments, Metrics, Latency, Storage, And Ablation

Unit Goal

This unit explains how the prototype is evaluated. It focuses on measurable results instead of unsupported claims.

Learning Objectives

After this unit, the student should be able to:

1. Explain latency measurement.
2. Explain storage overhead.
3. Explain tamper detection results.
4. Explain metadata exposure.
5. Explain experiment modes.
6. Explain policy ablation.
7. Understand why every claim must come from an artifact.

Topics

4.1 Latency And Storage Overhead

Latency means time taken.

Main file:

```
prototype/synthetic_access_sim/measure_overhead.py
```

Command:

```
python3 prototype/synthetic_access_sim/measure_overhead.py \  
--run-id unit4_latency_storage
```

Important outputs:

```
prototype/runs/unit4_latency_storage/artifacts/latency_summary.csv  
prototype/runs/unit4_latency_storage/artifacts/latency_samples.csv  
prototype/runs/unit4_latency_storage/artifacts/storage_overhead.csv  
prototype/runs/unit4_latency_storage/artifacts/overhead_comparison.csv
```

The prototype measures:

- policy decision time;
- explanation generation time;

- mutable log write time;
- signed hash-chain write time;
- signed hash-chain verification time;
- blockchain-style block creation time;
- blockchain-style verification time;
- storage overhead.

Important boundary:

This is local prototype timing. It is not a real deployment benchmark.

4.2 Experiment Modes

Main file:

```
prototype/synthetic_access_sim/experiment_modes.py
```

Command:

```
python3 prototype/synthetic_access_sim/experiment_modes.py \  
--run-id unit4_experiment_modes
```

Important output:

```
prototype/results/tables/experiment_modes_step6_comparison.csv
```

Experiment modes compare simpler and stronger designs. This prevents the paper from claiming the proposed method is useful without comparing it with baselines.

4.3 Policy Ablation

Ablation means removing or weakening one component to see what changes.

Main file:

```
prototype/synthetic_access_sim/policy_ablation.py
```

Command:

```
python3 prototype/synthetic_access_sim/policy_ablation.py \  
--input-run-id unit2_policy_seed42 \  
--run-id unit4_policy_ablation_seed42
```

Important output:

```
prototype/results/tables/policy_ablation_step8_comparison.csv
```

Example ablations:

- remove sensitivity rule;

- remove approval rule;
- remove jurisdiction rule;
- weaken purpose rule;
- compare false allows and false denies.

4.4 Research Metrics

Important metrics in this prototype include:

Metric	Meaning
Tamper detection rate	How often a defense detects altered logs.
False allow	A request is allowed when it should not be.
False deny	A request is denied when it should not be.
Escalation rate	How often requests need higher approval.
Latency	Time taken for decision, logging, or verification.
Storage overhead	Extra storage added by audit structures.
Metadata exposure	Risk that non-content fields reveal sensitive patterns.
AAS	Attack-aware score across attack scenarios.

4.5 Evidence Discipline

The repository follows an evidence rule:

Do not claim what the artifacts do not show.

Strong research claims must come from:

- CSV result tables;
- JSON metrics;
- plots;
- logs;
- tests;
- source-backed literature notes.

4.6 Tiny Technical Details To Learn

- A single seed is not enough for a strong result.
- Baselines are necessary.
- Ablations are necessary.
- Timing results depend on local environment.
- Policy ablations help show which security rule matters.
- Results must include limitations.

4.7 Hands-On Work

1. Run latency/storage measurement.

2. Open `latency_summary.csv`.
3. Run experiment modes.
4. Open comparison table.
5. Run policy ablation.
6. Write one safe claim based only on a table.

4.8 Student Notes To Prepare

Create this table:

Experiment	File/script	Output table	What it proves	What it does not prove
------------	-------------	--------------	----------------	------------------------

4.9 Viva Questions

1. What is latency?
2. What is storage overhead?
3. What is ablation?
4. Why do we need baselines?
5. What is a false allow?
6. Why are false allows serious in sensitive records?
7. Why should we avoid unsupported claims?

Unit 5: NS-PI, ADV-AUDIT, Attack Catalog, Baselines, And Research Evidence

Unit Goal

This unit explains the newer research package that moves the project beyond a simple prototype. It covers the attack benchmark, baseline defenses, NS-PI model, drift detection, counterfactual XAI, and the next research step.

Learning Objectives

After this unit, the student should be able to:

1. Explain the purpose of `src/seba/`.
2. Understand immutable schema design.
3. Explain the ADV-AUDIT attack catalog.
4. Explain baseline defenses.
5. Understand AAS scoring.
6. Explain NS-PI and drift detection.
7. Explain the current honest result.
8. Explain why `compromised_signer` is the next key experiment.

Topics

5.1 Research Package Structure

Main package:

```
src/seba/
```

Important folders:

```
src/seba/schema.py
src/seba/attacks/
src/seba/baselines/
src/seba/nsapi/
src/seba/scoring/
tests/
scripts/
```

5.2 Schema And Immutability

Main file:

```
src/seba/schema.py
```

Test file:

```
tests/test_schema.py
```

The schema uses immutable data structures. Attacks should return new modified logs instead of secretly changing original logs.

This matters because reproducible experiments require clean input and output separation.

5.3 ADV-AUDIT Attack Catalog

Attack files:

```
src/seba/attacks/backdate.py
src/seba/attacks/replay.py
src/seba/attacks/revocation_race.py
src/seba/attacks/explanation_swap.py
src/seba/attacks/metadata_inference.py
src/seba/attacks/collusion.py
src/seba/attacks/adaptive.py
src/seba/attacks/catalog.py
```

Attack examples:

- timestamp backdating;
- token replay;
- revocation race;
- explanation hash swap;
- metadata inference;
- validator collusion;

- adaptive attack.

The attack catalog gives external ground truth. This prevents the model from evaluating itself in a circular way.

5.4 Baseline Defenses

Baseline files:

```
src/seba/baselines/ct_log.py
src/seba/baselines/fabric_abac.py
```

Other detector logic appears in:

```
src/seba/scoring/detectors.py
```

Baseline ideas:

- mutable log;
- signed chain;
- blockchain-style audit;
- CT-log-style transparency log;
- Fabric+ABAC-style baseline;
- ABAC re-execution.

Strong baselines are required. The research should not compare only with weak systems.

5.5 AAS Scoring And Full Grid

Scoring files:

```
src/seba/scoring/aas.py
src/seba/scoring/grid.py
```

Command:

```
python3 scripts/run_full_grid.py
```

Outputs:

```
results/tables/full_grid_raw.csv
results/tables/full_grid_per_attack.csv
results/tables/full_grid_aas_by_defense.csv
```

The full grid runs:

```
defense x attack x seed
```

This is stronger than testing one method on one scenario.

5.6 NS-PI Model

NS-PI means Neuro-Symbolic Policy Induction.

Important files:

```
src/seba/nspi/learner.py
src/seba/nspi/drift.py
src/seba/nspi/counterfactual.py
```

NS-PI has three main parts:

1. Rule-list learner - learns interpretable policy behavior from access traces.
2. Drift detector - checks whether observed decisions differ from expected policy behavior.
3. Counterfactual generator - explains what minimal change could change a decision.

5.7 Current Honest Result

The current evidence shows:

- cryptographic defenses detect current row-tampering attacks very well;
- NS-PI drift does not beat cryptographic defenses on the current attack catalog;
- this is because most current attacks change logged fields and break hashes;
- NS-PI is better framed as a complementary signal, not a replacement for cryptographic audit.

Important file:

```
results/FINDINGS.md
```

5.8 Next Required Experiment: Compromised Signer

The most important next experiment is:

```
src/seba/attacks/compromised_signer.py
```

This attack should simulate a situation where:

1. the attacker changes access decisions;
2. the attacker re-signs the chain using a valid signer key;
3. cryptographic chain verification passes;
4. NS-PI may still detect suspicious decision-behavior drift.

Expected useful result:

```
Cryptographic detectors miss the re-signed corrupted log, while NS-PI detects a policy-distrib
```

If this does not happen, the paper should be reframed as an ADV-AUDIT benchmark paper instead of claiming NS-PI is a stronger method.

5.9 Paper Evidence Pack

Important files:

```
scripts/prepare_paper_evidence_pack.py
papers/final_paper/results/experiment_results_narrative.md
results/tables/paper_evidence_index.csv
results/tables/paper_table_01_method_comparison.csv
results/tables/paper_table_02_tamper_detection.csv
results/tables/paper_table_03_metadata_exposure.csv
results/tables/paper_table_04_latency_storage.csv
results/tables/paper_table_05_policy_ablation.csv
```

Command:

```
python3 scripts/prepare_paper_evidence_pack.py
```

5.10 Tiny Technical Details To Learn

- NS-PI is not predicting criminals.
- NS-PI learns policy behavior from access traces.
- Drift detection is not the same as hash-chain verification.
- Counterfactual explanations must avoid leaking sensitive policy bypass information.
- The current main claim is not fully proven.
- The next experiment decides whether the paper is a method paper or a benchmark paper.

5.11 Hands-On Work

1. Read `results/FINDINGS.md`.
2. Run `python3 scripts/run_full_grid.py`.
3. Open `full_grid_aas_by_defense.csv`.
4. Read `src/seba/nspi/learner.py`.
5. Read `src/seba/nspi/drift.py`.
6. Read `src/seba/nspi/counterfactual.py`.
7. Write a short note on why `compromised_signer` is needed.

5.12 Student Notes To Prepare

Write a one-page research update:

```
What has been implemented?
What has been tested?
What result is strong?
What result is weak?
What is the next experiment?
What paper direction is safest?
```

5.13 Viva Questions

1. What is ADV-AUDIT?
2. What is AAS?
3. What is NS-PI?
4. What does drift detection measure?

5. Why does NS-PI not beat hash chains on simple tampering?
6. What is a compromised signer?
7. Why is the compromised-signer attack important?
8. What is the safest publishable contribution if NS-PI does not win?

Complete Practical Lab List

Lab	Command or task	Expected output
Lab 1	<code>make test</code>	Verify test suite passes.
Lab 2	Run synthetic request generator	Synthetic CSV and JSON artifacts.
Lab 3	Run policy oracle	Labeled requests and explanation artifacts.
Lab 4	Inspect one decision	Simple allow/deny/escalate explanation.
Lab 5	Run audit baseline	Mutable log and signed hash-chain results.
Lab 6	Run blockchain audit	Permissioned audit blocks and validator set.
Lab 7	Run off-chain storage simulation	Pointer and metadata-exposure artifacts.
Lab 8	Run latency/storage measurement	Latency and storage comparison tables.
Lab 9	Run policy ablation	Policy ablation comparison table.
Lab 10	Run full evaluation grid	AAS result tables.
Lab 11	Read NS-PI code	Explain learner, drift, and counterfactual modules.
Lab 12	Design compromised-signer experiment	Written experiment plan.

Full 5-Unit Schedule

Unit	Title	Suggested Hours	Main Deliverable
1	Prototype Foundation, Research Scope, And Synthetic Data	8	One-page problem and synthetic data explanation.
2	Policy Validation, Access Control, And XAI	10	Request-to-decision trace with explanation.
3	Audit Logs, Hash Chains, Blockchain Layer, And Off-Chain Storage	12	Three audit diagrams and off-chain flow explanation.
4	Experiments, Metrics, Latency, Storage, And Ablation	10	Experiment-metric table with one safe claim.
5	NS-PI, ADV-AUDIT, Attack Catalog, Baselines, And Research Evidence	12	Research update and compromised-signer experiment note.
Total		52	Complete prototype understanding.

Final Student Checklist

The student should be able to:

- ☐ Explain the full SEBA-XAI flow.
- ☐ Generate synthetic requests.
- ☐ Validate requests with the policy oracle.
- ☐ Explain allow, deny, and escalate.
- ☐ Explain where XAI appears.
- ☐ Explain mutable logs.
- ☐ Explain signed hash chains.
- ☐ Explain permissioned blockchain-style audit.
- ☐ Explain why raw records stay off-chain.
- ☐ Run latency and storage experiments.
- ☐ Run policy ablation.
- ☐ Explain ADV-AUDIT attacks.
- ☐ Explain baseline defenses.
- ☐ Explain AAS scoring.
- ☐ Explain NS-PI.
- ☐ Explain drift detection.
- ☐ Explain counterfactual XAI.
- ☐ Explain why the current NS-PI claim is not fully proven.
- ☐ Explain the next `compromised_signer` experiment.

Final Professor-Ready Summary

The SEBA-XAI prototype is a synthetic research system for secure, explainable, and blockchain-audited access governance. It generates synthetic police-style access requests, validates them using deterministic policy rules, creates explanations, stores audit commitments, compares mutable logs with signed hash chains and permissioned blockchain-style audit, models off-chain storage, and measures latency, storage, tamper detection, metadata exposure, and policy ablation effects.

The newer research package adds an attack catalog, baseline defenses, AAS scoring, and NS-PI, an interpretable policy-induction method. The current evidence shows that cryptographic defenses already detect simple log tampering strongly, while NS-PI is not yet proven as superior. The next important experiment is a compromised-signer attack, where the audit chain remains valid but the decision behavior becomes suspicious. That experiment will decide whether the paper should be framed as an NS-PI method paper or an ADV-AUDIT benchmark paper.